Student Learning Management System

Low Level Design Document

Upgrade2Architect Phase 3 — Version 1.0

Java Spring Boot | React | AWS ECS Fargate | PostgreSQL | Redis | DynamoDB | S3

Attribute	Value
Document title	Student LMS — Low Level Design
Version	1.0
Status	Draft
Technology	Java 21, Spring Boot 3.x, React/Next.js, Docker, AWS ECS Fargate
Author	Pratyush Chandra Jha - 2154615
Date	22-May-2026
Parent document	HLD_Student-Learning-Management-System_2154615_v1.0

Table of Contents

1. Introduction

1.1 Purpose

This Low Level Design (LLD) document provides the detailed technical specification for the Student Learning Management System. It covers the database schema, class diagrams, sequence diagrams, component internals, API contracts, and deployment topology. This document is intended to guide developers in implementing the system and to allow the Architecture Review Board (ARB) to validate design decisions at the code and infrastructure level.

1.2 Scope

This document covers the following LLD artefacts:

- Entity-Relationship (ER) diagram — all 8 database tables with columns, types, and relationships
- Class diagram — Spring Boot domain model with fields, methods, and associations
- Sequence diagrams — quiz submission flow and course enrolment flow
- Component diagram — internal layered structure of each Spring Boot microservice
- Deployment view — AWS VPC topology, subnets, and service placement
- API design — all REST endpoints with request/response schemas and error codes
- Data access patterns — query strategies, caching, and indexing decisions
- Error handling — exception hierarchy and response schema
- State machines — course status and quiz attempt lifecycle

2. Database Design

2.1 ER diagram summary

The relational schema is hosted on Amazon RDS PostgreSQL (Multi-AZ). The following 8 tables form the core data model. All primary keys use auto-increment integers. Foreign key constraints enforce referential integrity. Soft-delete patterns (status flags) are used instead of hard deletes to preserve audit history.

2.2 Table specifications

Users

Column	Type	Constraints	Notes
UserId	INT	PK, AUTO_INCREMENT	Surrogate key
Username	VARCHAR(50)	NOT NULL, UNIQUE	Login identifier
Password	VARCHAR(255)	NOT NULL	BCrypt hash (cost 12)
Email	VARCHAR(100)	NOT NULL, UNIQUE	Used for notifications
Role	VARCHAR(20)	NOT NULL	ENUM: Admin | Instructor | Student
CreatedAt	DATETIME	NOT NULL DEFAULT NOW()	Audit timestamp
IsActive	BOOLEAN	NOT NULL DEFAULT TRUE	Soft-delete flag

Courses

Column	Type	Constraints	Notes
CourseId	INT	PK, AUTO_INCREMENT	Surrogate key
Title	VARCHAR(200)	NOT NULL	Course display name
Description	TEXT	NULLABLE	Rich text description
Category	VARCHAR(50)	NOT NULL	e.g. Programming, Design
Difficulty	VARCHAR(20)	NOT NULL	ENUM: Beginner | Intermediate | Advanced
InstructorId	INT	FK → Users(UserId)	Creator / owner
Status	VARCHAR(20)	NOT NULL DEFAULT Draft	ENUM: Draft | PendingApproval | Published | Archived

Column	Type	Constraints	Notes
Price	DECIMAL(10,2)	NOT NULL DEFAULT 0.00	0.00 = free
Duration	INT	NOT NULL	Estimated hours

Modules

Column	Type	Constraints	Notes
ModuleId	INT	PK, AUTO_INCREMENT	
CourseId	INT	FK → Courses(CourseId), CASCADE DELETE	
Title	VARCHAR(200)	NOT NULL	
OrderIndex	INT	NOT NULL	Sequence within course

Lessons

Column	Type	Constraints	Notes
LessonId	INT	PK, AUTO_INCREMENT	
ModuleId	INT	FK → Modules(ModuleId), CASCADE DELETE	
Title	VARCHAR(200)	NOT NULL	
ContentType	VARCHAR(20)	NOT NULL	ENUM: Video | Text | PDF
ContentUrl	VARCHAR(255)	NOT NULL	S3 URL or pre-signed URL
Duration	INT	NULLABLE	Minutes (for video lessons)
OrderIndex	INT	NOT NULL	Sequence within module

Enrollments

Column	Type	Constraints	Notes
EnrollmentId	INT	PK, AUTO_INCREMENT	
StudentId	INT	FK → Users(UserId)	Must have Role=Student
CourseId	INT	FK → Courses(CourseId)	Must be Published

Column	Type	Constraints	Notes
EnrolledAt	DATETIME	NOT NULL DEFAULT NOW()	
CompletionPercentage	DECIMAL(5,2)	NOT NULL DEFAULT 0.00	0.00–100.00
CompletedAt	DATETIME	NULLABLE	Set when CompletionPct = 100. Triggers cert gen

Assessments

Column	Type	Constraints	Notes
AssessmentId	INT	PK, AUTO_INCREMENT	
CourseId	INT	FK → Courses(CourseId)	
Title	VARCHAR(200)	NOT NULL	
Type	VARCHAR(20)	NOT NULL	ENUM: Quiz | Assignment
MaxScore	DECIMAL(6,2)	NOT NULL	
TimeLimitMinutes	INT	NULLABLE	NULL = untimed
MaxAttempts	INT	NOT NULL DEFAULT 1	Enforced in Assessment service

Submissions

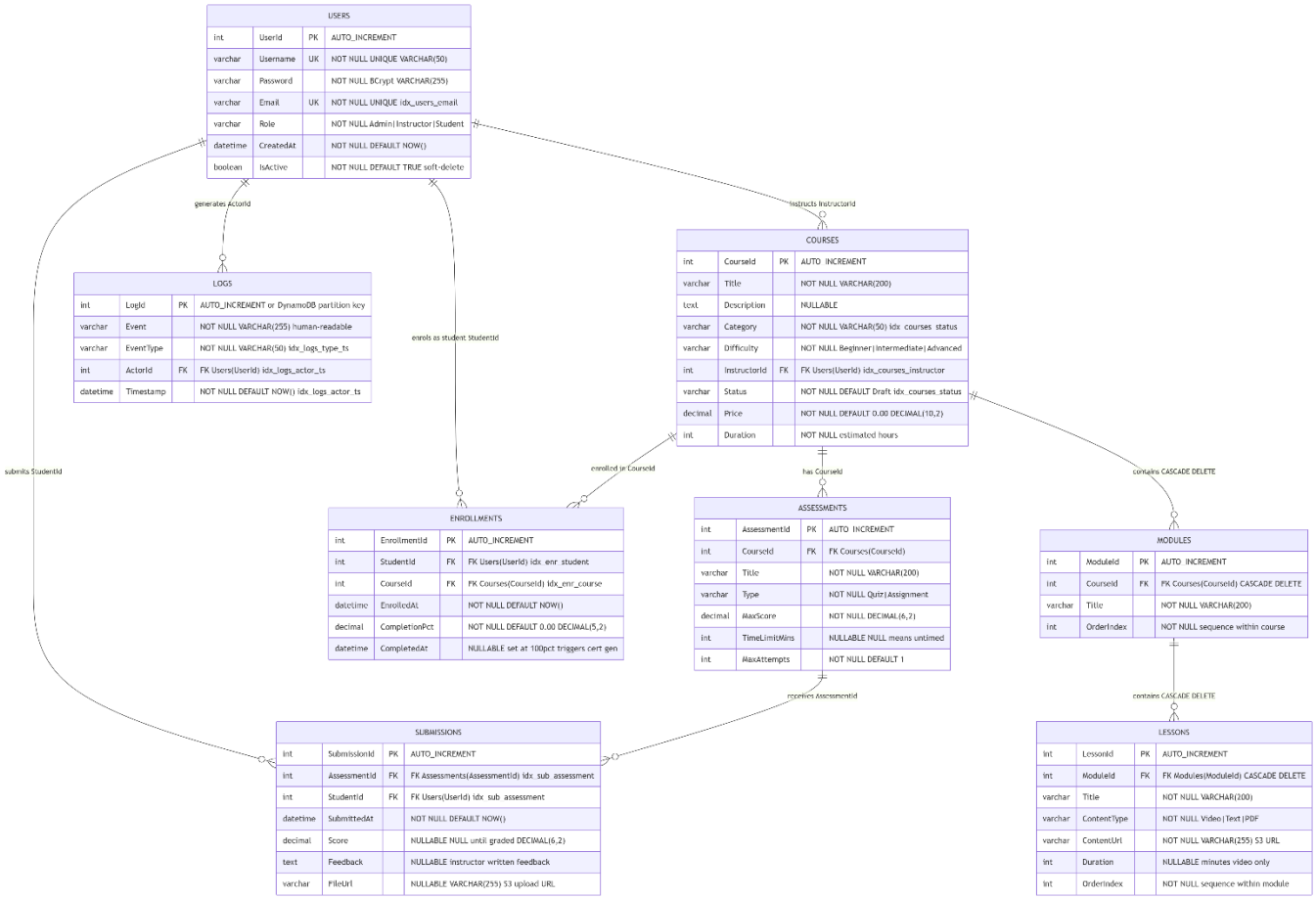
Column	Type	Constraints	Notes
SubmissionId	INT	PK, AUTO_INCREMENT	
AssessmentId	INT	FK → Assessments(AssessmentId)	
StudentId	INT	FK → Users(UserId)	
SubmittedAt	DATETIME	NOT NULL DEFAULT NOW()	
Score	DECIMAL(6,2)	NULLABLE	NULL until graded. Auto-set for MCQ/TF
Feedback	TEXT	NULLABLE	Instructor written feedback
FileUrl	VARCHAR(255)	NULLABLE	S3 URL for assignment file uploads

Logs

Column	Type	Constraints	Notes
LogId	INT	PK (or DynamoDB partition key)	
Event	VARCHAR(255)	NOT NULL	Human-readable description
EventType	VARCHAR(50)	NOT NULL	ENUM: Login | Enrolment | LessonView | QuizAttempt | Submission | Error
ActorId	INT	FK → Users(UserId)	
Timestamp	DATETIME	NOT NULL DEFAULT NOW()	Index on Timestamp for range queries

2.3 Indexes

Table	Index	Columns	Purpose
Users	idx_users_email	email	Unique lookup on login
Courses	idx_courses_instructor	InstructorId	Filter courses by instructor
Courses	idx_courses_status	Status, Category	Catalogue search and filter
Enrollments	idx_enr_student	StudentId	Student dashboard queries
Enrollments	idx_enr_course	CourseId	Cohort progress reports
Submissions	idx_sub_assessment	AssessmentId, StudentId	Attempt count check
Logs	idx_logs_actor_ts	ActorId, Timestamp	Admin filtered log queries
Logs	idx_logs_type_ts	EventType, Timestamp	Event type filtered queries


USERS
int UserId PK AUTO_INCREMENT
varchar Username UK NOT NULL UNIQUE VARCHAR(50)
varchar Password NOT NULL BCrypt VARCHAR(255)
varchar Email UK NOT NULL UNIQUE idx_users_email
varchar Role NOT NULL Admin|Instructor|Student
datetime CreatedAt NOT NULL DEFAULT NOW()
boolean IsActive NOT NULL DEFAULT TRUE soft-delete
generates ActorId
instructs InstructorId
LOGS
int LogId PK AUTO_INCREMENT or DynamoDB partition key
varchar Event NOT NULL VARCHAR(255) human-readable
varchar EventType NOT NULL VARCHAR(50) idx_logs_type_ts
int ActorId FK FK Users(UserId) idx_logs_actor_ts
datetime Timestamp NOT NULL DEFAULT NOW() idx_logs_actor_ts
COURSES
int CourseId PK AUTO_INCREMENT
varchar Title NOT NULL VARCHAR(200)
text Description NULLABLE
varchar Category NOT NULL VARCHAR(50) idx_courses_status
varchar Difficulty NOT NULL Beginner|Intermediate|Advanced
int InstructorId FK FK Users(UserId) idx_courses_instructor
varchar Status NOT NULL DEFAULT Draft idx_courses_status
decimal Price NOT NULL DEFAULT 0.00 DECIMAL(10,2)
int Duration NOT NULL estimated hours
enrols as student StudentId
submits StudentId
enrolled in CourseId
has CourseId
contains CASCADE DELETE
ENROLLMENTS
int EnrollmentId PK AUTO_INCREMENT
int StudentId FK FK Users(UserId) idx_enr_student
int CourseId FK FK Courses(CourseId) idx_enr_course
datetime EnrolledAt NOT NULL DEFAULT NOW()
decimal CompletionPct NOT NULL DEFAULT 0.00 DECIMAL(5,2)
datetime CompletedAt NULLABLE set at 100pct triggers cert gen
ASSESSMENTS
int AssessmentId PK AUTO_INCREMENT
int CourseId FK FK Courses(CourseId)
varchar Title NOT NULL VARCHAR(200)
varchar Type NOT NULL Quiz|Assignment
decimal MaxScore NOT NULL DECIMAL(6,2)
int TimeLimitMins NULLABLE NULL means untimed
int MaxAttempts NOT NULL DEFAULT 1
receives AssessmentId
MODULES
int ModuleId PK AUTO_INCREMENT
int CourseId FK FK Courses(CourseId) CASCADE DELETE
varchar Title NOT NULL VARCHAR(200)
int OrderIndex NOT NULL sequence within course
contains CASCADE DELETE
SUBMISSIONS
int SubmissionId PK AUTO_INCREMENT
int AssessmentId FK FK Assessments(AssessmentId) idx_sub_assessment
int StudentId FK FK Users(UserId) idx_sub_assessment
datetime SubmittedAt NOT NULL DEFAULT NOW()
decimal Score NULLABLE NULL until graded DECIMAL(6,2)
text Feedback NULLABLE instructor written feedback
varchar FileUrl NULLABLE VARCHAR(255) S3 upload URL
LESSONS
int LessonId PK AUTO_INCREMENT
int ModuleId FK FK Modules(ModuleId) CASCADE DELETE
varchar Title NOT NULL VARCHAR(200)
varchar ContentType NOT NULL Video|Text|PDF
varchar ContentUrl NOT NULL VARCHAR(255) S3 URL
int Duration NULLABLE minutes video only
int OrderIndex NOT NULL sequence within module

3. Class Diagram

3.1 Domain model

The Spring Boot domain model follows a standard JPA entity pattern. Each entity class maps directly to a database table. Enumerations are used for constrained string fields. Service classes encapsulate business logic. Repository interfaces extend JpaRepository for CRUD operations.

3.2 Core entity classes

User entity

```
@Entity @Table(name="users")
public class User {
  @Id @GeneratedValue(strategy=IDENTITY) private Integer userId;
  @Column(unique=true, nullable=false) private String username;
  @Column(nullable=false) private String passwordHash;
  @Column(unique=true, nullable=false) private String email;
  @Enumerated(EnumType.STRING) @Column(nullable=false) private Role role;
  @Column(nullable=false) private boolean isActive = true;
  // getters, setters, equals, hashCode
}
```

Course entity

```
@Entity @Table(name="courses")
public class Course {
  @Id @GeneratedValue(strategy=IDENTITY) private Integer courseId;
  @Column(nullable=false) private String title;
  @ManyToOne @JoinColumn(name="InstructorId") private User instructor;
  @Enumerated(EnumType.STRING) private CourseStatus status;
  @Enumerated(EnumType.STRING) private Difficulty difficulty;
  @OneToMany(mappedBy="course", cascade=CascadeType.ALL) private List<Module> modules;
  public void publish() { this.status = CourseStatus.PENDING_APPROVAL; }
  public void approve() { this.status = CourseStatus.PUBLISHED; }
  public void archive() { this.status = CourseStatus.ARCHIVED; }
}
```

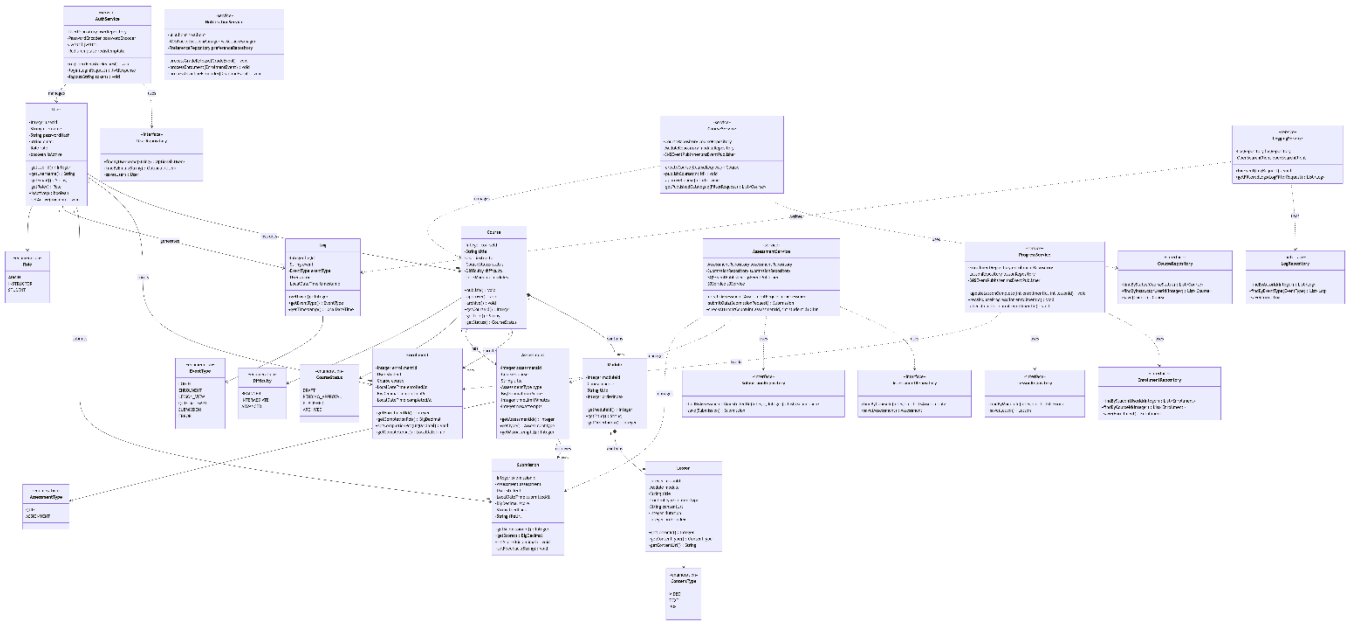
Enumerations

Enum	Values
Role	ADMIN, INSTRUCTOR, STUDENT
CourseStatus	DRAFT, PENDING_APPROVAL, PUBLISHED, ARCHIVED
Difficulty	BEGINNER, INTERMEDIATE, ADVANCED
ContentType	VIDEO, TEXT, PDF

Enum	Values
AssessmentType	QUIZ, ASSIGNMENT
EventType	LOGIN, ENROLMENT, LESSON_VIEW, QUIZ_ATTEMPT, SUBMISSION, ERROR

3.3 Service layer classes

Service class	Key methods	Dependencies
AuthService	register(RegisterRequest), login(LoginRequest) → JwtResponse, logout(String token)	UserRepository, PasswordEncoder, JwtUtil, RedisTemplate
CourseService	createCourse(CourseRequest), publishCourse(int id), approveCourse(int id), getPublishedCatalogue(FilterRequest)	CourseRepository, ModuleRepository, SNSEventPublisher
AssessmentService	createAssessment(AssessmentRequest), submitQuiz(SubmissionRequest), checkAttemptCount(int assessmentId, int studentId)	AssessmentRepository, SubmissionRepository, SQSEventPublisher, S3Service
ProgressService	updateLessonComplete(int enrollmentId, int lessonId), recalculateProgress(int enrollmentId), checkCompletion(int enrollmentId)	EnrollmentRepository, LessonRepository, SNSEventPublisher
NotificationService	processGradeRelease(GradeEvent), processEnrolment(EnrolmentEvent), processDeadlineReminder(DeadlineEvent)	SESClient, WebSocketSessionManager, PreferenceRepository
LoggingService	logEvent(LogRequest), getFilteredLogs(LogFilterRequest)	LogRepository, OpenSearchClient



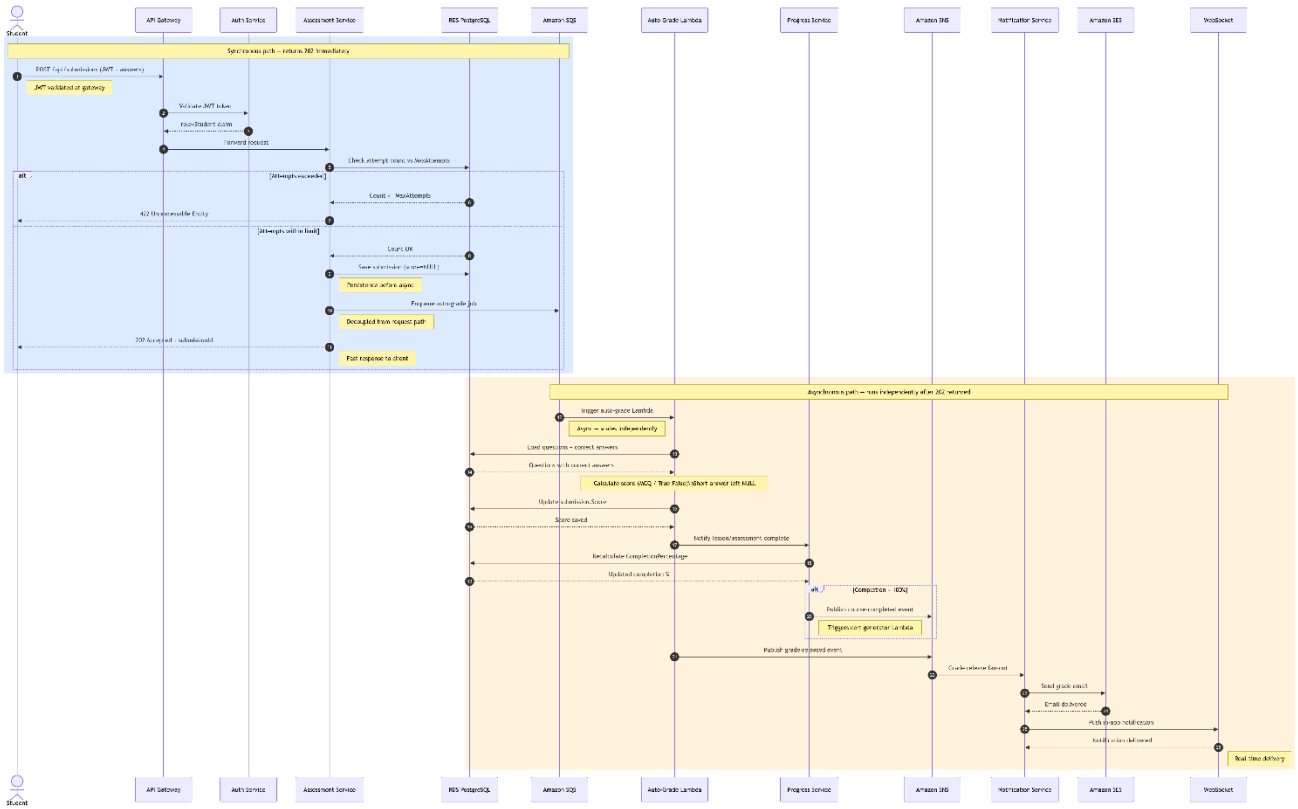
4. Sequence Diagrams

4.1 Quiz submission flow

The quiz submission flow is the most complex business process in the system, involving synchronous API calls, asynchronous grading, progress updates, certificate generation, and multi-channel notifications. The key design decision is that the API returns 202 Accepted immediately after persisting the submission, while all downstream processing happens asynchronously.

Step	Actor	Action	Notes
1	Student → API Gateway	POST /api/submissions with JWT + answers	JWT validated at gateway
2	API Gateway → Auth service	Validate JWT token	Returns role=Student claim
3	API Gateway → Assessment service	Forward request	
4	Assessment service → RDS	Check attempt count vs MaxAttempts	Reject if exceeded
5	Assessment service → RDS	Save submission with score=NULL	Persistence before async
6	Assessment service → SQS	Enqueue auto-grade job	Decoupled from request path
7	Assessment service → Student	202 Accepted + submissionId	Fast response to client
8	SQS → Lambda	Trigger auto-grade Lambda	Async; scales independently
9	Lambda → RDS	Load questions + correct answers	
10	Lambda	Calculate score (MCQ/True-False)	Short answer left NULL
11	Lambda → RDS	Update submission.Score	
12	Lambda → Progress service	Notify lesson/assessment complete	
13	Progress service → RDS	Recalculate CompletionPercentage	
14	Progress service (if 100%)	Publish course-completed event to SNS	Triggers cert generator Lambda
15	Lambda → SNS	Publish grade-released event	
16	SNS → Notification service	Grade release fan-out	
17	Notification service → SES	Send grade email	

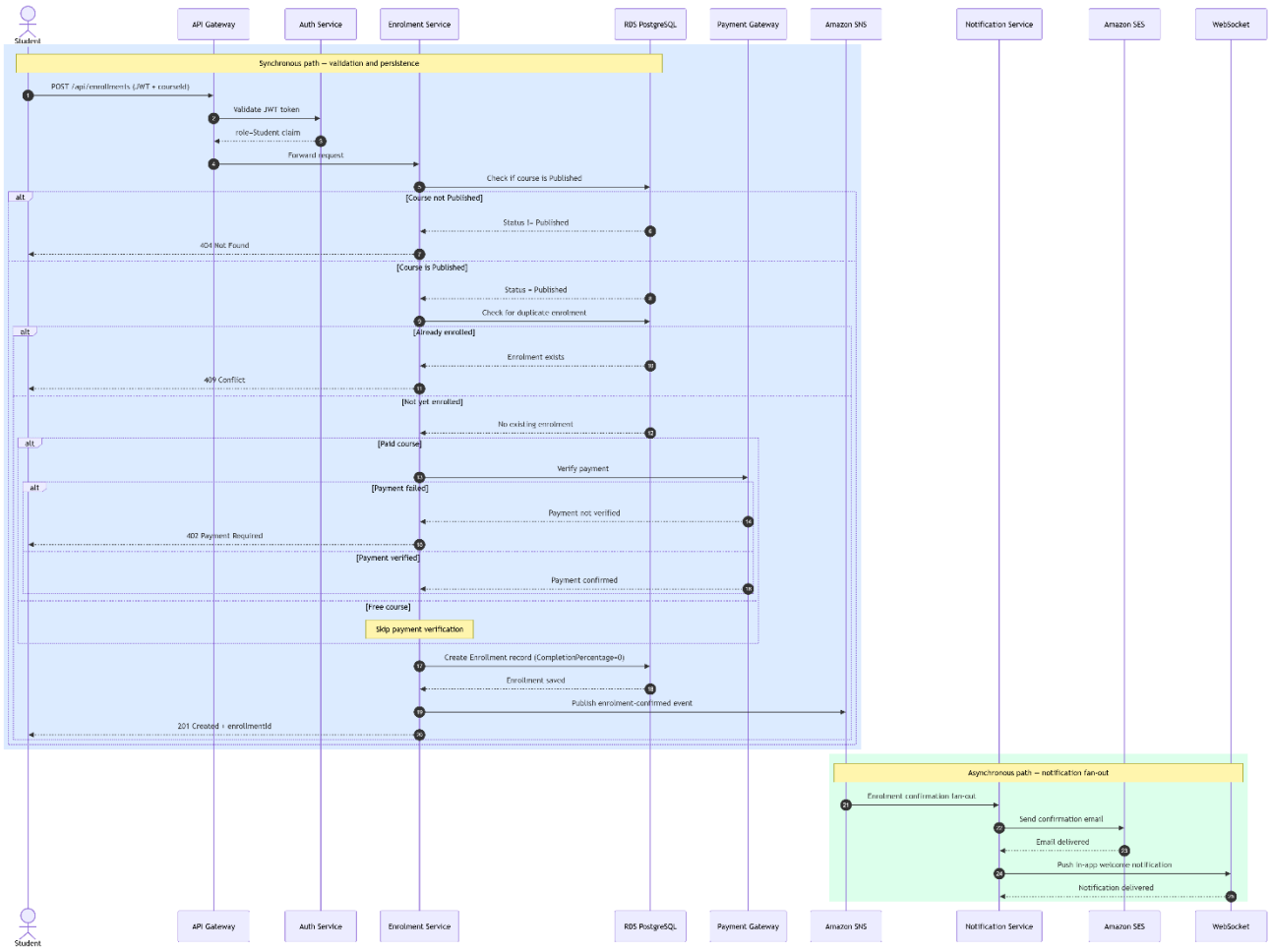
Step	Actor	Action	Notes
18	Notification service → WebSocket	Push in-app notification	Real-time delivery



4.2 Course enrolment flow

Step	Actor	Action	Notes
1	Student → API Gateway	POST /api/enrollments with JWT + courseId	
2	API Gateway → Auth service	Validate JWT (role=Student)	
3	API Gateway → Enrolment service	Forward request	
4	Enrolment service → RDS	Check if course is Published	Reject if not Published
5	Enrolment service → RDS	Check for duplicate enrolment	Reject if already enrolled
6	Enrolment service (paid course)	Verify payment with Payment Gateway	Skip for free courses

Step	Actor	Action	Notes
7	Enrolment service → RDS	Create Enrollment record	CompletionPercentage = 0
8	Enrolment service → SNS	Publish enrolment-confirmed event	
9	SNS → Notification service	Enrolment confirmation fan-out	
10	Notification service → SES	Send confirmation email	
11	Notification service → WebSocket	Push in-app welcome notification	
12	Enrolment service → Student	201 Created + enrollmentId	



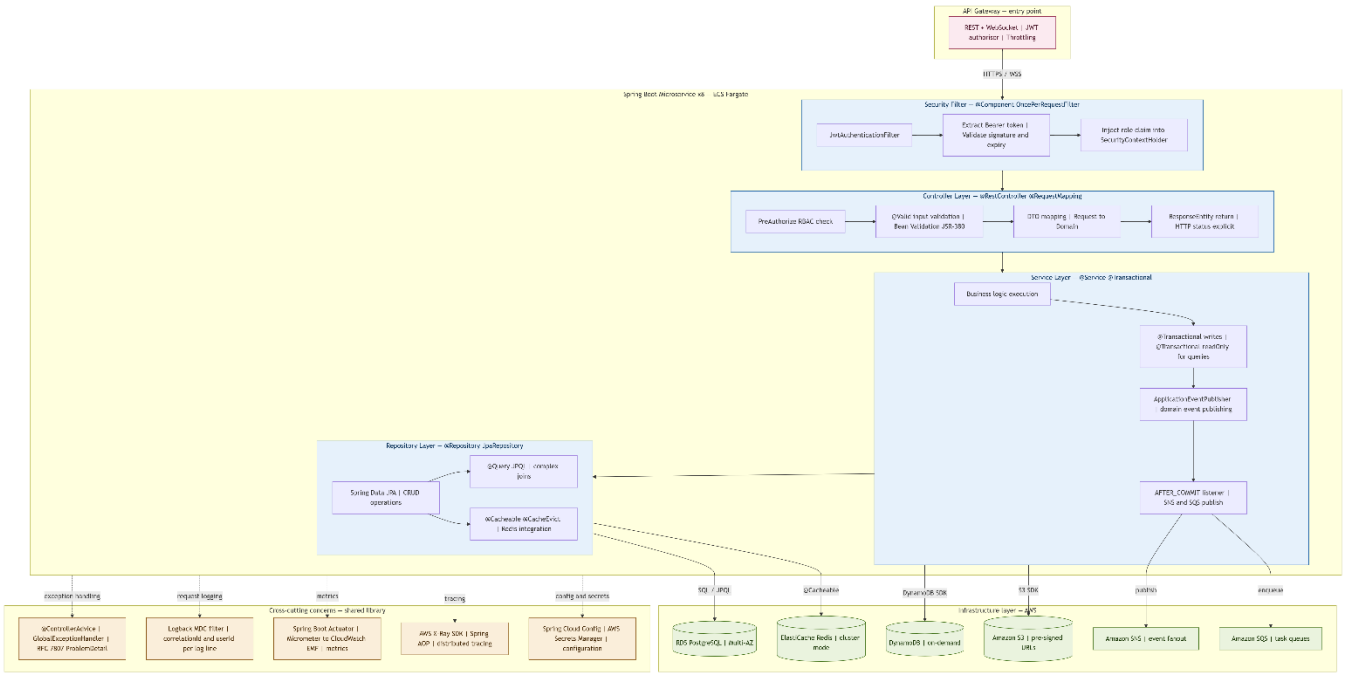
5. Component Diagram

5.1 Spring Boot service internals

Every one of the 8 microservices follows the same 4-layer architecture. This consistency ensures that any developer can navigate any service without learning a new structure. The layers are enforced by Spring Boot annotations and package naming conventions.

Layer	Annotation	Responsibility	Key patterns
Controller	@RestController, @RequestMapping	HTTP request handling, input validation via @Valid, DTO mapping	@PreAuthorize for RBAC; ResponseEntity return types; @ExceptionHandler for local errors
Service	@Service, @Transactional	Business logic, transaction boundaries, domain event publishing	@Transactional(readOnly=true) for queries; @Transactional for writes; ApplicationEventPublisher for domain events

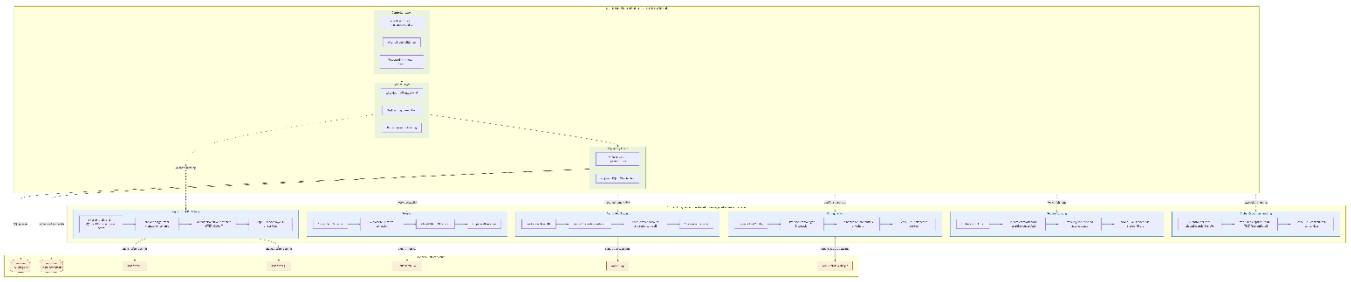
Layer	Annotation	Responsibility	Key patterns
Repository	@Repository, JpaRepository	Data access, JPQL queries, cache integration	Spring Data JPA for CRUD; @Query for complex joins; @Cacheable / @CacheEvict for Redis integration
Security filter	@Component (OncePerRequestFilter)	JWT extraction and validation, RBAC claim injection	JwtAuthenticationFilter; SecurityContextHolder for downstream RBAC checks



5.2 Cross-cutting concerns

Concern	Implementation	Scope
Global exception handling	@ControllerAdvice GlobalExceptionHandler — maps exceptions to RFC 7807 Problem Detail responses	All controllers in all services
Request logging	Logback MDC filter — injects correlationId, userId, requestPath into every log line	All services via shared library
Async event publishing	SNSEventPublisher / SQSEventPublisher Spring beans — fire-and-forget after transaction commit	Service layer in all services
Metrics	Spring Boot Actuator + Micrometer → CloudWatch EMF	All services
Distributed tracing	AWS X-Ray Java SDK + Spring AOP — traces every controller and repository call	All services

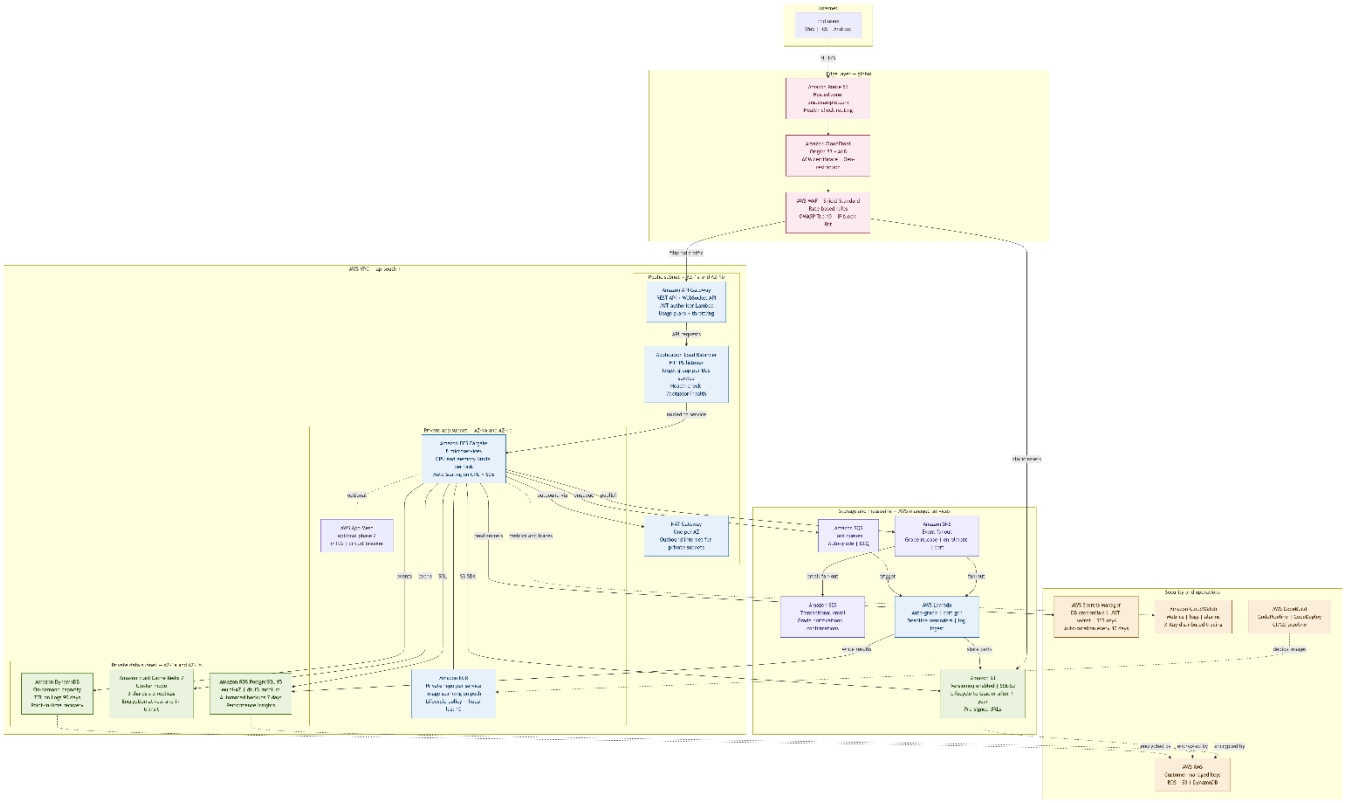
Concern	Implementation	Scope
Configuration	Spring Cloud Config + AWS Secrets Manager — no hardcoded credentials	All services at startup



6. Deployment View

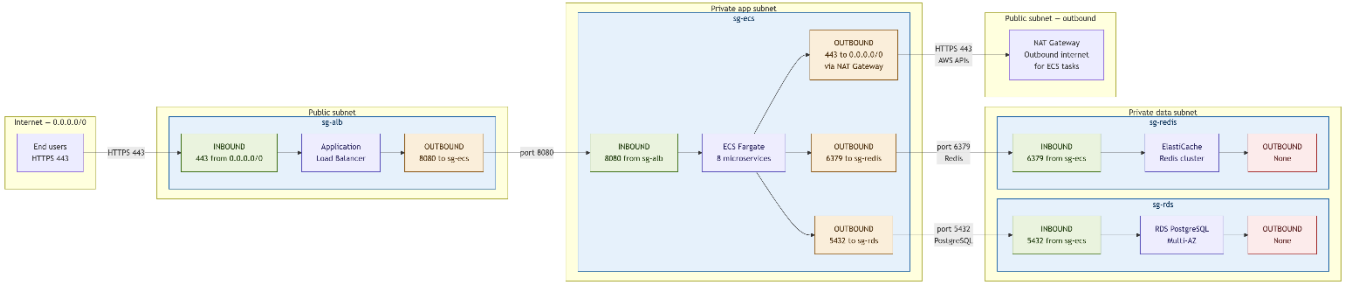
6.1 AWS infrastructure topology

Layer	AWS resource	Configuration
DNS	Amazon Route 53	Hosted zone for lms.example.com; health-check routing to CloudFront
CDN / edge	Amazon CloudFront	Origin: S3 (static assets) + ALB (API); WAF attached; ACM certificate; Geo-restriction optional
DDoS / WAF	AWS WAF + Shield Standard	Rate-based rules; AWS managed rule groups (OWASP Top 10); IP block list
API entry point	Amazon API Gateway	Regional REST API + WebSocket API; JWT authoriser Lambda; usage plans with throttling
Load balancing	Application Load Balancer	HTTPS listener; target groups per ECS service; health check /actuator/health
Container orchestration	Amazon ECS Fargate	8 services; task definitions with CPU/memory limits; Service Auto Scaling on CPU > 60%
Container registry	Amazon ECR	Private repository per service; image scanning on push; lifecycle policy (keep last 10 images)
Relational DB	Amazon RDS PostgreSQL 15	Multi-AZ; db.t3.medium (scalable to r6g); automated backups 7 days; Performance Insights
Cache	Amazon ElastiCache Redis 7	Cluster mode; 2 shards x 2 replicas; encryption at rest and in transit
NoSQL	Amazon DynamoDB	On-demand capacity; TTL on Logs table (90 days); Point-in-time recovery
Object store	Amazon S3	Versioning enabled; SSE-S3 encryption; lifecycle rules (Glacier after 1 year); pre-signed URLs
Secrets	AWS Secrets Manager	DB credentials, JWT secret, SES keys; automatic rotation every 30 days
Encryption keys	AWS KMS	Customer managed keys for RDS, S3, DynamoDB
Outbound internet	NAT Gateway	One per AZ; ECS tasks in private subnet route outbound via NAT
Service mesh	AWS App Mesh (optional phase 2)	mTLS between services; traffic management; circuit breaker



6.2 Network security groups

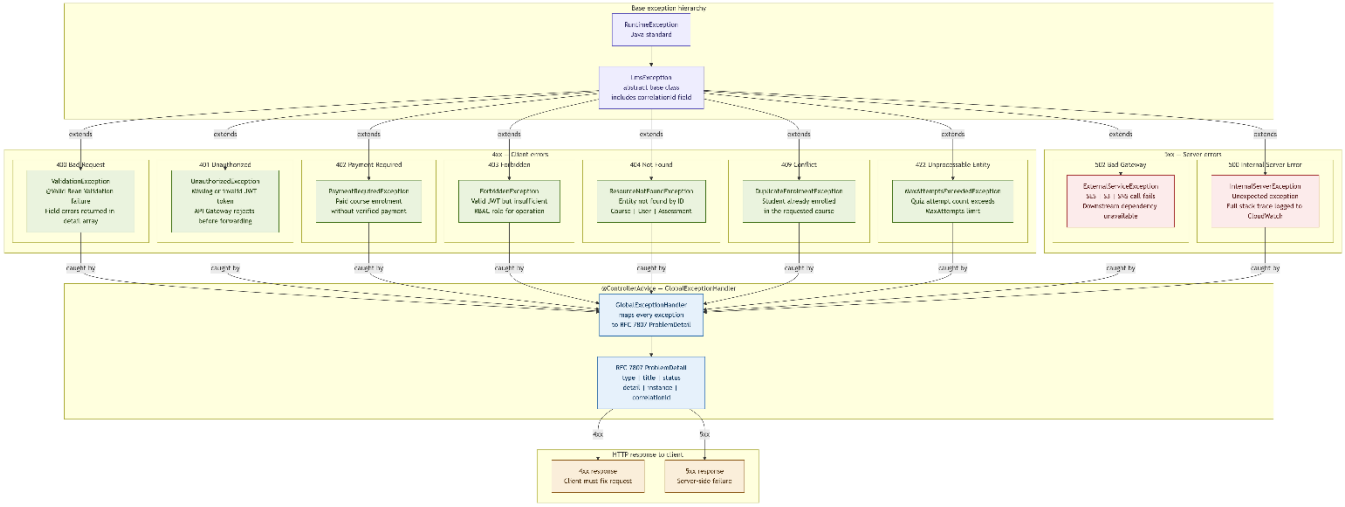
Security group	Inbound	Outbound
sg-alb	443 from 0.0.0.0/0	8080 to sg-ecs
sg-ecs	8080 from sg-alb	5432 to sg-rds; 6379 to sg-redis; 443 to 0.0.0.0/0 (via NAT)
sg-rds	5432 from sg-ecs	None
sg-redis	6379 from sg-ecs	None



7. Error Handling Design

7.1 Exception hierarchy

Exception class	HTTP status	When thrown
ResourceNotFoundException	404 Not Found	Entity not found by ID (course, user, assessment)
UnauthorizedException	401 Unauthorized	Missing or invalid JWT token
ForbiddenException	403 Forbidden	Valid token but insufficient RBAC permissions
ValidationException	400 Bad Request	Bean Validation failure (@Valid)
DuplicateEnrolmentException	409 Conflict	Student attempts to enrol in already-enrolled course
MaxAttemptsExceededException	422 Unprocessable Entity	Quiz attempt exceeds MaxAttempts limit
PaymentRequiredException	402 Payment Required	Student attempts to enrol in paid course without payment
ExternalServiceException	502 Bad Gateway	SES, S3, or SNS call fails
InternalServerException	500 Internal Server Error	Unexpected exception; logged with full stack trace



7.2 Standard error response schema (RFC 7807)

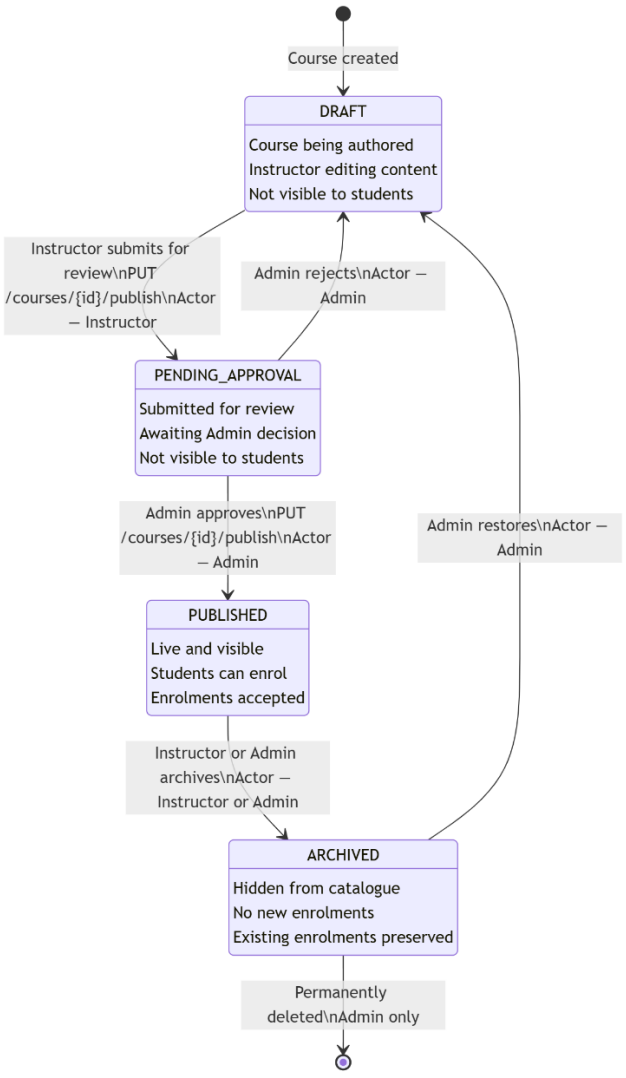
```
{
  "type": "https://lms.example.com/errors/resource-not-found",
  "title": "Resource not found",
  "status": 404,
```

```
  "detail": "Course with ID 42 does not exist",
  "instance": "/api/courses/42",
  "correlationId": "a3f2-bc9d-11ef"
}
```

8. State Machines

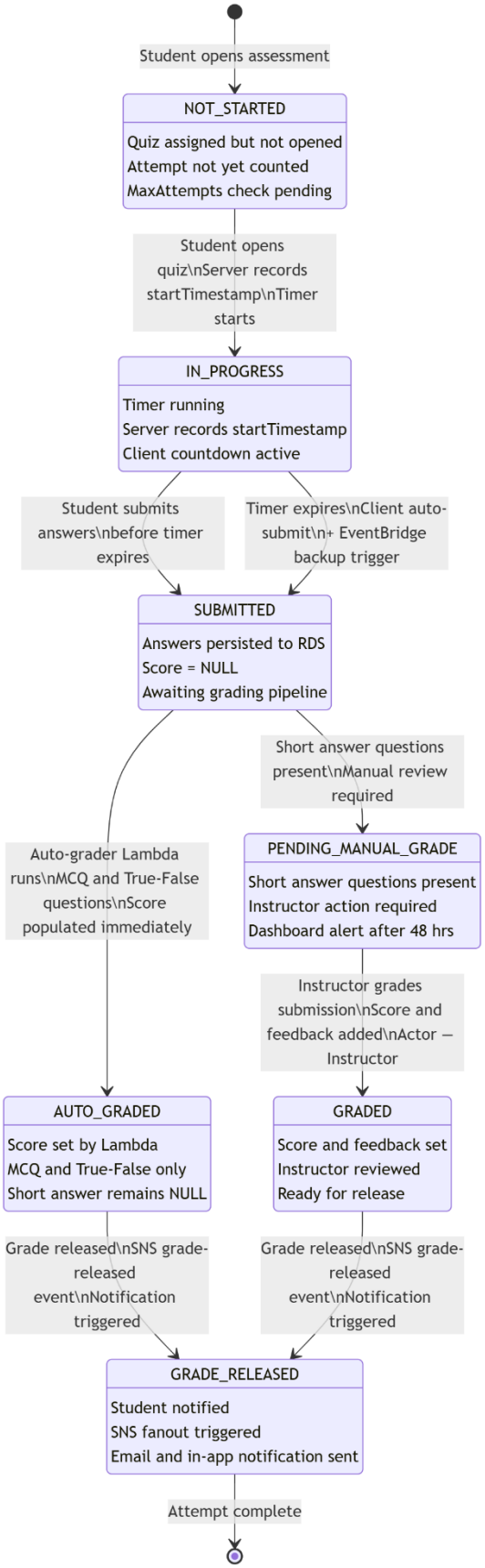
8.1 Course lifecycle

Current state	Event	Next state	Actor
DRAFT	Instructor submits for review	PENDING_APPROVAL	Instructor (PUT /courses/{id}/publish)
PENDING_APPROVAL	Admin approves	PUBLISHED	Admin (PUT /courses/{id}/publish)
PENDING_APPROVAL	Admin rejects	DRAFT	Admin
PUBLISHED	Instructor or Admin archives	ARCHIVED	Instructor / Admin
ARCHIVED	Admin restores	DRAFT	Admin



8.2 Quiz attempt lifecycle

Current state	Event	Next state	Notes
NOT_STARTED	Student opens quiz	IN_PROGRESS	Timer starts; server records start timestamp
IN_PROGRESS	Student submits answers	SUBMITTED	Before timer expires
IN_PROGRESS	Timer expires	SUBMITTED	Auto-submit triggered by client + server-side EventBridge backup
SUBMITTED	Auto-grader runs (MCQ/TF)	AUTO_GRADED	Score populated immediately
SUBMITTED	Manual review needed	PENDING_MANUAL_GRADE	Short answer questions
PENDING_MANUAL_GRADE	Instructor grades	GRADED	Score + feedback added
AUTO_GRADED or GRADED	Grade released	GRADE_RELEASED	Notification triggered


Student opens assessment
NOT_STARTED
Quiz assigned but not opened
Attempt not yet counted
MaxAttempts check pending
Student opens quiz\nServer records startTimestamp\nTimer starts
IN_PROGRESS
Timer running
Server records startTimestamp
Client countdown active
Student submits answers\nbefore timer expires
Timer expires\nClient auto-submit\n+ EventBridge backup trigger
SUBMITTED
Answers persisted to RDS
Score = NULL
Awaiting grading pipeline
Auto-grader Lambda runs\nMCQ and True-False questions\nScore populated immediately
Short answer questions present\nManual review required
PENDING_MANUAL_GRADE
Short answer questions present
Instructor action required
Dashboard alert after 48 hrs
Instructor grades submission\nScore and feedback added\nActor — Instructor
AUTO_GRADED
Score set by Lambda
MCQ and True-False only
Short answer remains NULL
GRADED
Score and feedback set
Instructor reviewed
Ready for release
Grade released\nSNS grade-released event\nNotification triggered
Grade released\nSNS grade-released event\nNotification triggered
GRADE_RELEASED
Student notified
SNS fanout triggered
Email and in-app notification sent
Attempt complete

9. API Design Summary

Full API design with request/response schemas, authentication headers, error codes, and versioning is documented in the companion API Design Document. The table below provides a quick reference of all endpoints grouped by domain.

Method	Endpoint	Service	Auth	Notes
POST	/api/auth/register	Auth	Public	Register new user
POST	/api/auth/login	Auth	Public	Returns JWT access + refresh tokens
POST	/api/auth/logout	Auth	JWT	Invalidates refresh token
GET	/api/users	User	Admin	List all users
GET	/api/users/{id}	User	JWT	Get user by ID
PUT	/api/users/{id}	User	JWT	Update profile
DELETE	/api/users/{id}	User	Admin	Deactivate user
GET	/api/courses	Course	Public	Published catalogue
GET	/api/courses/{id}	Course	Public	Course detail
POST	/api/courses	Course	Instructor	Create course
PUT	/api/courses/{id}	Course	Instructor	Update course
DELETE	/api/courses/{id}	Course	Instructor/Admin	Archive course
PUT	/api/courses/{id}/publish	Course	Instructor/Admin	Submit/approve
GET	/api/courses/{id}/modules	Course	JWT	List modules
POST	/api/modules	Course	Instructor	Create module
PUT	/api/modules/{id}	Course	Instructor	Update module
DELETE	/api/modules/{id}	Course	Instructor	Delete module
GET	/api/modules/{id}/lessons	Course	JWT	List lessons
POST	/api/lessons	Course	Instructor	Create lesson
PUT	/api/lessons/{id}	Course	Instructor	Update lesson
DELETE	/api/lessons/{id}	Course	Instructor	Delete lesson
POST	/api/enrollments	Enrolment	Student	Enrol in course
GET	/api/enrollments?student_id={id}	Enrolment	JWT	Student's courses
GET	/api/enrollments?course_id={id}	Enrolment	Instructor/Admin	Course roster

Method	Endpoint	Service	Auth	Notes
DELETE	/api/enrollments/{id}	Enrolment	Student/Admin	Unenrol
GET	/api/assessments?course_id={id}	Assessment	JWT	Course assessments
POST	/api/assessments	Assessment	Instructor	Create assessment
PUT	/api/assessments/{id}	Assessment	Instructor	Update assessment
DELETE	/api/assessments/{id}	Assessment	Instructor	Delete assessment
POST	/api/submissions	Assessment	Student	Submit quiz/assignment
GET	/api/submissions?assessment_id={id}	Assessment	Instructor/Admin	All submissions
GET	/api/submissions/{id}	Assessment	JWT	Single submission
PUT	/api/submissions/{id}/grade	Assessment	Instructor	Grade submission
GET	/api/progress?student_id={id}&course_id={id}	Progress	JWT	Progress detail
PUT	/api/progress/lesson-complete	Progress	Student	Mark lesson done
GET	/api/logs	Logging	Admin	Filtered log list
GET	/api/logs/{id}	Logging	Admin	Single log entry
POST	/api/logs	Logging	System	Internal log write

10. Unit Testing Strategy

10.1 Coverage requirements

Per F011, API code coverage must exceed 80%. The following components require mandatory unit tests:

- Quiz grading logic (auto-grade Lambda and AssessmentService.autoGrade())
- Progress calculation (ProgressService.recalculateProgress())
- Certificate generation trigger (ProgressService.checkCompletion())
- JWT token generation and validation (JwtUtil)
- RBAC enforcement (SecurityConfig and @PreAuthorize expressions)

10.2 Test framework

Test type	Framework	Scope
Unit tests	JUnit 5 + Mockito	Service layer business logic; all methods in grading, progress, cert trigger
Integration tests	Spring Boot Test + Testcontainers (PostgreSQL, Redis)	Repository layer; full service stack with real DB
API tests	MockMvc (Spring Boot Test)	Controller layer; all endpoints; request/response validation
E2E smoke tests	Postman / Newman in CI	Full flow: register → enrol → submit → grade → certificate
Code coverage	JaCoCo (fail if < 80%)	Configured in Maven/Gradle build; gates CodeBuild step

10.3 Test naming convention

```
@Test
void methodName_givenCondition_thenExpectedOutcome() { ... }
```

Example: progressService_givenAllLessonsComplete_thenCompletionPctIs100()

11. Glossary

Term	Definition
DTO	Data Transfer Object — a plain object used to transfer data between layers, avoiding exposing JPA entities directly to the API layer
JPA	Jakarta Persistence API — Java standard for ORM (Object-Relational Mapping), used via Hibernate in Spring Boot
JPQL	Jakarta Persistence Query Language — object-oriented query language for JPA entities, analogous to SQL
MockMvc	Spring Boot test utility for testing MVC controllers without starting a full HTTP server
MDC	Mapped Diagnostic Context — thread-local key-value store in Logback for enriching log lines with contextual data
RFC 7807	IETF standard for HTTP API error responses — defines the Problem Detail JSON format
Testcontainers	Java library that spins up real Docker containers (PostgreSQL, Redis) during integration tests
TTL	Time To Live — DynamoDB attribute that marks items for automatic deletion after a specified timestamp (used for log retention)